

January 2026 CSE 214: Software Engineering Sessional

Assignment on Creational Design Pattern

Due: July 12, 2026 at 11:59 PM

1 Introduction

Modern software systems often need to create objects with a mixture of required fields, optional fields, default values, and validation rules. If this construction logic is handled only through long constructors or many overloaded constructors, the code quickly becomes hard to read and easy to misuse.

In this assignment, you will work with **FoodFlow**, a small online food ordering system. The system is already implemented and runnable, but its order creation logic has several design issues. Your task is to identify the most appropriate design improvement and refactor the code while preserving all existing behavior.

2 Project Scenario

FoodFlow is a command-line food ordering application for a small restaurant. It supports the following features:

- Loading menu items from a CSV file.
- Displaying the menu.
- Adding customized food items to a cart.
- Creating pickup, delivery, scheduled, and sample family orders.
- Applying coupon and loyalty discounts.
- Calculating service charges such as delivery, rush order, and gift wrapping.
- Generating a text receipt.

The goal of the assignment is not to add new business features. The goal is to improve the internal design of the object creation process.

3 Running the Project

Compile the project from the root directory:

Linux, macOS, or Git Bash:

```
javac -d out src/**/*.java TestHarness.java
```

Windows CMD:

```
if not exist out mkdir out
if exist sources.txt del sources.txt
for /R src %f in (*.java) do @echo "%f" >> sources.txt
echo TestHarness.java >> sources.txt
javac -d out @sources.txt
```

The quotes in `sources.txt` are important if the project folder path contains spaces, such as `Offline 1 CD`.

Before compiling, you can check that the JDK is installed by running:

```
java -version
javac -version
```

If `java` works but `javac` is not recognized, install JDK 8 or newer and add it to the system `PATH`. The starter code is compatible with Java 8.

Run the functional test harness:

```
java -cp out TestHarness
```

Run the application:

```
java -cp out Main data/menu.csv
```

On Windows CMD, the application can also be run with a backslash path:

```
java -cp out Main data\menu.csv
```

4 Sample Commands

```
menu
add B01 2 LARGE cheese spicy
add D01 2 MEDIUM
cart
checkout delivery Alice 01700000000 House-12_Road-7_Dhanmondi
receipt out/receipt.txt
sample
exit
```

5 Code Structure

```
src/
  Main.java
model/
  DeliveryType.java
  MenuItem.java
  Order.java
  OrderItem.java
  PaymentMethod.java
  Size.java
io/
  CsvMenuLoader.java
  ReceiptWriter.java
service/
```

```
MenuCatalog.java
OrderService.java
ReceiptService.java
cli/
  Cli.java
  CommandHandler.java
util/
  MoneyUtils.java
  TextUtils.java
data/
  menu.csv
TestHarness.java
```

6 Design Issues

The current system works, but the implementation has object creation problems. These problems are concentrated around the `Order` and `OrderItem` creation flow.

6.1 Issue 1: Long Constructor with Many Optional Parameters

The `Order` class has a constructor with many parameters:

```
new Order(orderId, customerName, phone, deliveryType, deliveryAddress,
          paymentMethod, scheduledTime, couponCode, giftWrap,
          cutleryRequired, loyaltyPointsToRedeem, rushOrder,
          items, specialInstructions)
```

Several of these parameters are optional. Some are strings, some are booleans, and some may be `null`. This makes the call sites difficult to read and easy to break.

6.2 Issue 2: Repeated Default Values

Default choices such as `PaymentMethod.CASH`, empty coupon code, no gift wrapping, cutlery required, and no scheduled time are repeated in multiple places. If the default policy changes, the same change may need to be made in many files.

6.3 Issue 3: Fragile Parameter Ordering

The constructor contains adjacent parameters of the same type, especially strings and booleans. Accidentally swapping two arguments may still compile, but the program behavior becomes wrong.

6.4 Issue 4: Mixed Responsibilities

The constructor currently performs too many tasks: validating required data, normalizing optional values, deciding defaults, copying the item list, and storing the final fields. A better design should make these steps more explicit and easier to test.

7 Your Task

Refactor the codebase to improve the object creation flow for orders. You should decide which design approach best addresses the problems described above. Your refactoring should preserve all observable behavior. The test harness should pass before and after your changes.

At minimum, your solution should:

1. Remove or significantly reduce the need for long, hard-to-read Order constructor calls in application logic.
2. Handle optional configuration such as delivery address, payment method, scheduled time, coupon code, gift wrapping, cutlery, loyalty points, rush order, and special instructions in a clearer and safer way.
3. Keep required fields such as order id, customer name, phone, and items clear and difficult to omit accidentally.
4. Refactor `OrderService` so that order creation becomes readable and less error-prone.
5. Preserve validation rules. For example, a delivery order must still have a non-empty delivery address, and an order must still contain at least one item.
6. Preserve pricing behavior exactly: subtotal, discount, service charges, and total should remain unchanged.

8 Expected Refactoring Direction

You are free to choose the exact class structure and method names. The refactored code should make order creation easier to read, harder to misuse, and easier to change when new optional order settings are added later. Your design should also centralize default values and validation rules instead of scattering them across multiple call sites.

9 Constraints

- Do not remove existing user-facing functionality.
- Do not change the pricing formulas.
- Do not introduce external libraries.
- Keep the project runnable using the provided `javac` commands.
- The final code should pass `TestHarness.java`.

10 Testing

The provided `TestHarness.java` performs functional checks for:

- Menu loading.
- Customized item pricing.
- Delivery order pricing.
- Scheduled gift order pricing.
- Sample family order pricing.
- Receipt generation.
- Validation of invalid delivery orders.

Passing the harness does not automatically guarantee a good design. It only confirms that your refactoring preserved the expected behavior. Your design will also be evaluated manually.

11 Submission

Submit a ZIP archive containing the entire refactored `src/` directory. Ensure that your code compiles and runs correctly.

12 Grading Rubric

Criterion	Points	Details
Correctness and behavior preservation	20	Code compiles, test harness passes, pricing and validation behavior remain unchanged.
Appropriate design choice for object creation	40	The selected design clearly handles required fields, optional fields, defaults, validation, and final object creation.
Refactoring quality	25	Long constructor calls are removed from application logic, duplication is reduced, and call sites become readable.
Code clarity and maintainability	15	Naming, formatting, package organization, and encapsulation are clean.
Total	100	